

Initial Access Techniques

4/15/26 2:11 PM

Core Mindset & Attack Chain

Goal: Reconstruct the attack from initial access to impact. **Why:** Initial access malware is the entry point — simpler but heavily relies on social engineering, obfuscation, and quick-changing infrastructure.

Standard Attack Chain (Memorize This)

1. **Initial Access** — Maldoc, script, or link (phishing).
2. **Execution** — Macro (VBA) or script (PowerShell/JScript) runs.
3. **Payload Delivery** — Downloads or extracts next-stage malware (EXE/DLL).
4. **Persistence** — Scheduled task, registry, or LOLBin.
5. **C2 Communication** — Beacon, command, exfiltration (HTTP/TLS/SMTP).
6. **Impact** — Data theft, further payload, or control.

Key Insight: Early stages change rapidly and rely on user interaction. Later stages become more complex.

Safe Handling & Lab Setup

When: Every time you acquire a sample. **Why:** Prevent accidental execution or spread.

Essential Practices

- **ZIP + Password:** Always use infected. Prevents AV scanning and accidental run.
- **Rename Extension:** .exe → .exe.txt or .bin. Stops OS auto-execution.
- **Dedicated Setup:**
 - Separate analysis machine (never work/personal).
 - VMs (Flare VM recommended for Windows tools).
 - **Host-only** network (prevents lateral movement).
 - Snapshots: Clean → Pre-execution → During analysis.
- **OPSEC:** Uploading to public sandboxes (Triage, ANY.RUN) may tip off attackers. Prefer private groups when possible.

How Malware Is Delivered

Common Vectors:

- **Phishing Emails** (most common): Social engineering (fear, urgency, fake CEO/invoice).
 - Links (harder to scan) or attachments.
 - Encrypted ZIP with password in email (obfuscated: remove dashes, image, etc.).
- **Maldocs** (Word, Excel, OneNote):
 - Macros (enable content lure).
 - Template injection (loads malicious template from internet).
 - Exploits (rare, e.g., old Equation Editor).
- **Living-off-the-Land:** Uses built-in tools (PowerShell).
- **Other:** Malicious ads, SMS scams, supply-chain attacks.

Defense Layers: Email filtering, user education, EDR, least privilege, DNS/firewall blocking, network monitoring.

Threat Actors & Infrastructure

Threat Actors:

- **Nation-State (APTs):** Sophisticated, stealthy, long campaigns.
- **Criminal Groups:** Volume-based, commodity malware (AgentTesla, RedLine, SmokeLoader).

Commodity Malware: Pre-built, sold on markets (stealers, downloaders). Check abuse.ch for top families.

Infrastructure:

- Compromised web servers (hard to block — legitimate sites).
- Abused platforms (Discord, Telegram).
- Webshells (remote shell access).
- Proxy scripts (hide real C2).
- Panels (web dashboards for managing victims).

Why it works: Fast rotation, blending with normal traffic, obfuscation.

Maldoc Analysis Workflow

Tool: oledump.py (inspects Office document structure/streams).

Steps (in order)

1. **Verify file**
Bash
file sample.doc
2. **Inspect structure**
Bash
oledump.py sample.doc
 - Look for: M (macro), O (embedded object), large streams.
3. **Extract stream**
Bash
oledump.py -s <index> sample.doc | more # preview
oledump.py -s <index> -d sample.doc > payload.bin # dump
4. **Fix embedded PE** (if needed)
 - Open in hex editor.
 - Delete bytes before MZ (4D 5A).
 - Save as new file.
5. **Identify payload**
Bash
shasum -a 256 payload.bin
Upload hash or file to Triage/ANY.RUN.

Main takeaway: Maldocs are containers. Extract the real payload or macro.

Macro & Script Analysis

Macros (VBA):

- Entry points: AutoOpen, Document_Open.
- Common patterns: CreateObject("WScript.Shell"), .exec, PowerShell download.
- Obfuscation: Random variables, string splitting (*), .replace().

Scripts (JScript/PowerShell):

- Executed by cscript.exe / wscript.exe.
- Look for: Base64 blobs, large string arrays, encoded commands.

When to debug:

- Heavy obfuscation (no visible URL/command).
- NOT needed if sandbox or static strings already show behavior.

Debugging (Macros – VBA):

- Open in Word → Enable macros.
- Developer → Visual Basic.
- Set breakpoint on key lines (CreateObject, execution).
- Run → inspect variables (hover or Watch window).

Debugging (JScript):

- ```
cmd
cscript //X script.js
```
- Triggers Visual Studio JIT debugger.
  - Set breakpoints, continue, watch Locals window for deobfuscated strings/commands.

**Main takeaway:** Focus on extracting the final command/payload, not full reverse engineering.

## Obfuscation & Debugging

**Common Techniques:**

- Base64 / hex encoding.
- Random variable/function names.
- Junk code / false flags.
- String padding + replace before execution.

**When to debug:** Only when static analysis fails to reveal command, URL, or payload. **Goal of debugging:** Reveal real values in memory (e.g., final PowerShell command or DLL path).

## Network Analysis & Detection

**What you observe:**

- PowerShell/EXE downloads.
- Direct IP requests (no DNS).
- IP lookup (api.ipify.org) — geo/anti-analysis.
- POST beacons, encoded data.
- Exfiltration (passwords.txt, systeminfo).

**Tools:**

- **Wireshark:** Follow streams, inspect HTTP/TLS handshake.
- **Suricata:** Offline PCAP analysis, alerts on PE downloads, PowerShell, TLS certs.
  - JA3 / JA3s fingerprints (client/server behavior).
  - TLS certificate details (subject, issuer).
- **EveBox:** View Suricata alerts.

**Threat Hunting Mindset:**

- Single alert = weak.
- Chain = strong (download → DLLs → exfil).
- Correlate host (processes) + network (traffic).

**Main takeaway:** Even TLS leaves detectable patterns (certs, JA3, behavior sequences).

## Overall Triage Workflow

1. **Safe handling** — ZIP + infected, rename, VM (host-only).
2. **Initial inspection** — file, oledump.py.
3. **Extract** — Macro or embedded payload.
4. **Analyze execution** — Look for AutoOpen, CreateObject, PowerShell.
5. **Debug if needed** — Reveal hidden commands (VBA or cscript //X).
6. **Identify payload** — Hash, sandbox (Triage/ANY.RUN).
7. **Network analysis** — Wireshark + Suricata for C2, downloads, exfil.
8. **Correlate** — Process chain + traffic patterns.
9. **Document IOCs** — URLs, IPs, hashes, commands, family.

**Final Mindset:**

- Follow the chain: Delivery → Execution → Payload → C2 → Impact.
- Combine static + dynamic + network.
- Debugging is a last resort when obfuscation hides the command.
- Goal = extract IOCs and understand behavior quickly for detection/hunting.